Taller — Consultas Básicas en PostgreSQL

Trabajo en clase

Universidad de Pamplona
Facultad de Ingenierías y Arquitectura
Programa de Ingeniería de Sistemas
Bases de Datos I — 2026-2
Prf. Juan Alejandro Carrillo Jaimes

Estudiante: _____ **Código:** _____

Indicaciones

- El taller contiene **10 ejercicios prácticos** de consultas básicas sobre una sola tabla, basados en la base de datos `coffee_coop_db` de la cooperativa cafetera trabajada en clase.
- **Únicamente** se usan las cláusulas `SELECT ... FROM`, `AS`, `WHERE` (comparaciones, `AND`, `OR`, `NOT`, `BETWEEN`, `IN`, `LIKE` / `ILIKE`, `IS [NOT] NULL`) y `ORDER BY` (`ASC` / `DESC`).
- **No** se usan `JOIN`, `GROUP BY`, funciones de agregación, subconsultas, `DISTINCT` ni `LIMIT`. Sí se usan operaciones aritméticas entre columnas.
- Toda tabla se escribe con su esquema (`geo.`, `farm.`, `buy.`, `pay.`) y el alias `T1`; las columnas del `SELECT` se referencian con ese alias (`T1.first_name`).
- Los alias de columnas están en inglés y en formato *snake_case*.
- Cada literal se desarrolló en un archivo `ex_NN_x.sql` dentro de `content/coffee-coop/scripts/querys/01-sql-basic/`; en este documento se presenta la consulta y el resultado obtenido en PostgreSQL. Cuando la salida tiene muchas filas se muestran las primeras 10 y el total que reporta el cliente.

Tablas por esquema

Esquema	Tablas
`geo`	`departments`, `municipalities`
`farm`	`farmers`, `farms`, `plots`, `coffee_varieties`, `plot_varieties`, `certifications`, `farm_certifications`
`buy`	`collection_points`, `daily_prices`, `deliveries`, `quality_analyses`
`pay`	`settlements`, `payments`

Orden de ejecución lógico de una consulta

PostgreSQL no ejecuta la consulta en el orden en que se escribe. Por eso un alias definido en el `SELECT` sí puede usarse en el `ORDER BY`, pero no en el `WHERE`.

```
FROM  →  WHERE  →  SELECT  →  ORDER BY
tabla    filtra    columnas    ordena el
y alias  filas     y alias     resultado
```

Ejercicios

Ejercicio 1. SELECT y alias — Catálogo de departamentos

La coordinación regional necesita el listado oficial de departamentos (códigos DANE) con el que trabaja la cooperativa, presentado con nombres de columna claros para un reporte.

Tabla: `geo.departments`

Literal a)

a) Consulte todas las columnas y todas las filas de la tabla.

Consulta — `ex_01_a.sql`

```
-- Exercise 01 - a
-- Departments: select all columns and rows
SELECT
    T1.department_code,
    T1.department_name
FROM geo.departments AS T1;
```

Resultado obtenido en PostgreSQL

```
"department_code"  | "department_name"
------+-------------------
05    | Antioquia
08    | Atlántico
11    | Bogotá, D.C.
13    | Bolívar
15    | Boyacá
17    | Caldas
18    | Caquetá
19    | Cauca
20    | Cesar
23    | Córdoba
(33 rows)
```

Literal b)

b) Consulte únicamente el código y el nombre del departamento, renombrando las columnas como `code` y `department`.

Consulta — `ex_01_b.sql`

```
-- Exercise 01 - b
-- Departments: code and name with column aliases
SELECT
T1.department_code AS code,
T1.department_name AS department
FROM geo.departments AS T1;
```

Resultado obtenido en PostgreSQL

```
department_code | department_name
----------------+------------------
05              | Antioquia
08              | Atlántico
11              | Bogotá, D.C.
13              | Bolívar
15              | Boyacá
17              | Caldas
18              | Caquetá
19              | Cauca
20              | Cesar
23              | Córdoba
(33 rows)
```

Literal c)

c) Repita el literal b) ordenando el resultado alfabéticamente por el nombre del departamento.

Consulta — `ex_01_c.sql`

```
-- Exercise 01 - c
-- Departments: code and name ordered alphabetically by department name
SELECT
    T1.department_code AS code,
    T1.department_name AS department
FROM geo.departments AS T1
ORDER BY T1.department_name ASC;
```

Resultado obtenido en PostgreSQL

```
code | department
-----+--------------------------------------------------
91   | Amazonas
05   | Antioquia
81   | Arauca
88   | Archipiélago De San Andrés, Providencia Y Santa Catalina
08   | Atlántico
11   | Bogotá, D.C.
13   | Bolívar
15   | Boyacá
17   | Caldas
18   | Caquetá
(33 rows)
```

Ejercicio 2. WHERE con booleanos — Variedades de café

El área de asistencia técnica recomienda a los caficultores sembrar variedades resistentes a la roya. Necesita consultar el catálogo de variedades para preparar una charla.

Tabla: `farm.coffee_varieties`

Literal a)

a) Consulte el nombre, la especie y la descripción de las variedades resistentes a la roya, con los alias `variety`, `species` y `description`.

Consulta — `ex_02_a.sql`

```sql
-- Exercise 02 - a
-- Coffee varieties: rust resistant varieties
SELECT
    T1.variety_name AS variety,
    T1.species AS species,
    T1.description AS description
FROM farm.coffee_varieties AS T1
WHERE T1.is_rust_resistant = TRUE;
```

Resultado obtenido en PostgreSQL

```
variety    | species | description
-----------+---------+-----------------------------------------------------------------
Castillo   | arabica | Variedad colombiana resistente a la roya, la más sembrada del país.
Colombia   | arabica | Variedad compuesta resistente a la roya, antecesora de Castillo.
Cenicafé 1 | arabica | Variedad de porte bajo con resistencia a la roya.
Tabi       | arabica | Variedad de porte alto resistente a la roya.
Robusta    | robusta | Especie Coffea canephora, de mayor contenido de cafeína.
(5 rows)
```

Literal b)

b) Consulte las variedades no resistentes a la roya de la especie *arabica*, ordenadas por nombre de forma descendente.

Consulta — `ex_02_b.sql`

```
-- Exercise 02 - b
-- Coffee varieties: arabica species non-resistant to rust ordered descending
SELECT
    T1.variety_name AS variety,
    T1.species AS species,
    T1.description AS description
FROM farm.coffee_varieties AS T1
WHERE T1.is_rust_resistant = FALSE
  AND T1.species = 'arabica'
ORDER BY T1.variety_name DESC;
```

Resultado obtenido en PostgreSQL

```
variety       | species | description
--------------+---------+-------------------------------------------------------------
Típica        | arabica | Variedad tradicional de porte alto.
Pacamara      | arabica | Cruce de Pacas y Maragogipe, grano grande.
Maragogipe    | arabica | Variedad de grano grande.
Geisha        | arabica | Variedad de especialidad con perfil floral.
Caturra       | arabica | Mutación de Borbón de porte bajo y alta productividad.
Borbón Rosado | arabica | Variedad de especialidad de fruto rosado.
Borbón        | arabica | Variedad tradicional de buena calidad en taza.
(7 rows)
```

Ejercicio 3. Columnas calculadas — Primas de certificación

Los asociados preguntan cuánto dinero adicional recibirían por cada carga de café (125 kg) si su finca obtiene una certificación. La tabla guarda la prima por kilogramo en pesos colombianos.

Tabla: `farm.certifications`

Literal a)

a) Consulte el nombre de la certificación, el organismo que la emite y la prima por kg, con los alias `certification`, `issuer` y `premium_per_kg_cop`, ordenado de la prima más alta a la más baja.

Consulta — `ex_03_a.sql`

```
-- Exercise 03 - a
-- Certifications: premium per kg ordered from highest to lowest
SELECT
    T1.certification_name AS certification,
    T1.issuing_body AS issuer,
    T1.premium_per_kg AS premium_per_kg_cop
FROM farm.certifications AS T1
ORDER BY T1.premium_per_kg DESC;
```

Resultado obtenido en PostgreSQL

```
certification      | issuer                           | premium_per_kg_cop
-------------------+----------------------------------+-------------------
Orgánico           | Ente certificador orgánico       | 800.00
Bird Friendly      | Smithsonian Migratory Bird Center | 500.00
Fairtrade          | Fairtrade International          | 400.00
Nespresso AAA      | Nespresso / Rainforest Alliance  | 350.00
Rainforest Alliance | Rainforest Alliance              | 300.00
C.A.F.E. Practices | Starbucks / SCS Global Services  | 250.00
4C                 | 4C Services                      | 150.00
(7 rows)
```

Literal b)

b) Agregue una columna calculada con la prima por carga (`premium_per_kg * 125`) con el alias `premium_per_load_cop`.

Consulta — `ex_03_b.sql`

```
-- Exercise 03 - b
-- Certifications: calculated premium per load (125 kg)
SELECT
    T1.certification_name AS certification,
    T1.issuing_body AS issuer,
    T1.premium_per_kg AS premium_per_kg_cop,
    (T1.premium_per_kg * 125) AS premium_per_load_cop
FROM farm.certifications AS T1
ORDER BY T1.premium_per_kg DESC;
```

Resultado obtenido en PostgreSQL

```
certification       | issuer                           | premium_per_kg_cop |
premium_per_load_cop
--------------------+----------------------------------+--------------------+-------------------
--
Orgánico            | Ente certificador orgánico       | 800.00             | 100000.00
Bird Friendly       | Smithsonian Migratory Bird Center | 500.00             | 62500.00
Fairtrade           | Fairtrade International          | 400.00             | 50000.00
Nespresso AAA       | Nespresso / Rainforest Alliance  | 350.00             | 43750.00
Rainforest Alliance | Rainforest Alliance              | 300.00             | 37500.00
C.A.F.E. Practices  | Starbucks / SCS Global Services  | 250.00             | 31250.00
4C                  | 4C Services                      | 150.00             | 18750.00
(7 rows)
```

Literal c)

c) Muestre solo las certificaciones cuya prima por kg sea mayor o igual a 350 COP.

Consulta — `ex_03_c.sql`

```
-- Exercise 03 - c
-- Certifications: premium per kg greater than or equal to 350 COP
SELECT
    T1.certification_name AS certification,
    T1.issuing_body AS issuer,
    T1.premium_per_kg AS premium_per_kg_cop,
    (T1.premium_per_kg * 125) AS premium_per_load_cop
FROM farm.certifications AS T1
WHERE T1.premium_per_kg >= 350
ORDER BY T1.premium_per_kg DESC;
```

Resultado obtenido en PostgreSQL

```
certification | issuer                            | premium_per_kg_cop | premium_per_load_cop
--------------+-----------------------------------+--------------------+---------------------
Orgánico      | Ente certificador orgánico        | 800.00             | 100000.00
Bird Friendly | Smithsonian Migratory Bird Center | 500.00             | 62500.00
Fairtrade     | Fairtrade International           | 400.00             | 50000.00
Nespresso AAA | Nespresso / Rainforest Alliance   | 350.00             | 43750.00
(4 rows)
```

Ejercicio 4. WHERE con LIKE — Municipios de Norte de Santander

La cooperativa abrirá nuevos puntos de compra en Norte de Santander (código DANE 54) y necesita revisar los municipios del departamento.

Tabla: `geo.municipalities`

Literal a)

a) Consulte el código y el nombre de los municipios de Norte de Santander, con los alias `municipality_code` y `municipality`, ordenados alfabéticamente.

Consulta — `ex_04_a.sql`

```
-- Exercise 04 - a
-- Municipalities: Norte de Santander (54) ordered alphabetically
SELECT
    T1.municipality_code AS municipality_code,
    T1.municipality_name AS municipality
FROM geo.municipalities AS T1
WHERE T1.department_code = '54'
ORDER BY T1.municipality_name ASC;
```

Resultado obtenido en PostgreSQL

```
municipality_code | municipality
------------------+--------------
54003             | Ábrego
54051             | Arboledas
54099             | Bochalema
54109             | Bucarasica
54128             | Cáchira
54125             | Cácota
54172             | Chinácota
54174             | Chitagá
54206             | Convención
54001             | Cúcuta
(10 rows)
```

Literal b)

b) Del mismo departamento, consulte únicamente los municipios cuyo nombre comienza con *San*.

Consulta — `ex_04_b.sql`

```
-- Exercise 04 - b
-- Municipalities: Norte de Santander starting with 'San'
SELECT
    T1.municipality_code AS municipality_code,
    T1.municipality_name AS municipality
FROM geo.municipalities AS T1
WHERE T1.department_code = '54'
  AND T1.municipality_name LIKE 'San%'
ORDER BY T1.municipality_name ASC;
```

Resultado obtenido en PostgreSQL

```
municipality_code | municipality
------------------+--------------
54670             | San Calixto
54673             | San Cayetano
54680             | Santiago
(3 rows)
```

Literal c)

c) Consulte en todo el país los municipios cuyo nombre contiene la palabra *Pamplona,* sin importar mayúsculas o minúsculas (`ILIKE`).

Consulta — `ex_04_c.sql`

```
-- Exercise 04 - c
-- Municipalities: nationwide containing 'Pamplona' (case insensitive)
SELECT
    T1.municipality_code AS municipality_code,
    T1.municipality_name AS municipality
FROM geo.municipalities AS T1
WHERE T1.municipality_name ILIKE '%Pamplona%';
```

Resultado obtenido en PostgreSQL

```
municipality_code | municipality
------------------+--------------
54518             | Pamplona
(1 row)
```

Ejercicio 5. IS NULL y operadores lógicos — Puntos de compra

El área de operaciones está depurando el directorio de puntos de compra: quiere saber cuáles están cerrados y cuáles no tienen teléfono registrado.

Tabla: `buy.collection_points`

Literal a)

a) Consulte el nombre, la dirección y el código del municipio de los puntos de compra inactivos, con los alias `point`, `address` y `municipality_code`.

Consulta — `ex_05_a.sql`

```
-- Exercise 05 - a
-- Collection points: inactive collection points
SELECT
    T1.point_name AS point,
    T1.address_line AS address,
    T1.municipality_code AS municipality_code
FROM buy.collection_points AS T1
WHERE T1.is_active = FALSE;
```

Resultado obtenido en PostgreSQL

```
point                       | address             | municipality_code
----------------------------+---------------------+------------------
Punto de Compra El Banco 1  | Calle 17 # 34-54    | 47245
Punto de Compra Popayán 1   | Carrera 21 # 4-95   | 19001
Punto de Compra Apía 2      | Calle 9 # 14-56     | 66045
Punto de Compra Tello 1     | Calle 8 # 13-23     | 41799
Punto de Compra Turbo 1     | Avenida 32 # 34-61  | 05837
Punto de Compra Paicol 2    | Calle 2 # 30-84     | 41518
Punto de Compra Rionegro 1  | Carrera 31 # 59-80  | 68615
Punto de Compra Ulloa 1     | Calle 15 # 3-50     | 76845
Punto de Compra Nátaga 1    | Diagonal 30 # 7-59  | 41483
(9 rows)
```

Literal b)

b) Consulte los puntos de compra que no tienen teléfono registrado, ordenados por nombre.

Consulta — `ex_05_b.sql`

```
-- Exercise 05 - b
-- Collection points: missing phone number ordered by name
SELECT
    T1.point_name AS point,
    T1.address_Line AS address,
    T1.municipality_code AS municipality_code
FROM buy.collection_points AS T1
WHERE T1.phone_number IS NULL
ORDER BY T1.point_name ASC;
```

Resultado obtenido en PostgreSQL

```
point                       | address               | municipality_code
----------------------------+-----------------------+------------------
Punto de Compra Aipe 1      | Carrera 13 # 17-16    | 41016
Punto de Compra Anserma 1   | Calle 39 # 7-96       | 17042
Punto de Compra Apía 2      | Calle 9 # 14-56       | 66045
Punto de Compra Baraya 1    | Calle 1 # 19-77       | 41078
Punto de Compra Bochalema 1 | Carrera 25 # 47-89    | 54099
Punto de Compra Bosconia 1  | Carrera 26 # 26-43    | 20060
Punto de Compra Cajibío 1   | Carrera 15 # 32-45    | 19130
Punto de Compra Cisneros 1  | Calle 37 # 10-5       | 05190
Punto de Compra Coello 1    | Transversal 20 # 49-66 | 73200
Punto de Compra El Carmen 2 | Calle 3 # 34-88       | 54245
(10 rows)
```

Literal c)

c) Consulte los puntos que estén inactivos o que no tengan teléfono, mostrando también las columnas `phone_number` e `is_active`.

Consulta — `ex_05_c.sql`

```
-- Exercise 05 - c
-- Collection points: inactive OR missing phone number
SELECT
    T1.point_name AS point,
    T1.address_line AS address,
    T1.municipality_code AS municipality_code,
    T1.phone_number AS phone_number,
    T1.is_active AS is_active
FROM buy.collection_points AS T1
WHERE T1.is_active = FALSE
   OR T1.phone_number IS NULL;
```

Resultado obtenido en PostgreSQL

```
point                       | address               | municipality_code | phone_number |
is_active
----------------------------+-----------------------+-------------------+--------------+---------
-
Punto de Compra El Banco 1  | Calle 17 # 34-54      | 47245             | 6055026534   | false
Punto de Compra Popayán 1   | Carrera 21 # 4-95     | 19001             | 6057341141   | false
Punto de Compra Hatonuevo 1 | Calle 7 # 16-89       | 44378             |              | true
Punto de Compra Bosconia 1  | Carrera 26 # 26-43    | 20060             |              | true
Punto de Compra Cajibío 1   | Carrera 15 # 32-45    | 19130             |              | true
Punto de Compra Anserma 1   | Calle 39 # 7-96       | 17042             |              | true
Punto de Compra Apía 2      | Calle 9 # 14-56       | 66045             |              | false
Punto de Compra Puracé 1    | Calle 16 # 50-5       | 19585             |              | true
Punto de Compra Aipe 1      | Carrera 13 # 17-16    | 41016             |              | true
Punto de Compra Marsella 1  | Transversal 19 # 42-92 | 66440             |              | true
(10 rows)
```

Ejercicio 6. Filtros combinados — Asociados de Pamplona

La seccional de Pamplona (código DANE 54518) quiere convocar a sus asociadas a un programa de liderazgo femenino y actualizar los datos de contacto de quienes no tienen correo.

Tabla: `farm.farmers`

Literal a)

a) Consulte el documento, nombres, apellidos y teléfono de las asociadas (`gender = 'F'`) del municipio 54518, con los alias `document`, `first_name`, `last_name` y `phone`, ordenadas por apellido y luego por nombre.

Consulta — `ex_06_a.sql`

```
-- Exercise 06 - a
-- Farmers: female farmers in Pamplona (54518) ordered by last and first name
SELECT
    T1.document_number AS document,
    T1.first_name AS first_name,
    T1.last_name AS last_name,
    T1.phone_number AS phone
FROM farm.farmers AS T1
WHERE T1.municipality_code = '54518'
  AND T1.gender = 'F'
ORDER BY T1.last_name ASC, T1.first_name ASC;
```

Resultado obtenido en PostgreSQL

```
document   | first_name      | last_name        | phone
-----------+-----------------+------------------+-----------
768461420  | Marleny         | Agudelo Sandoval | 3242280898
931079279  | Edilma          | Álvarez Bonilla  | 3236902029
463842401  | Rocío           | Álvarez Cárdenas | 3047898372
788628125  | Mónica          | Álvarez Carrillo | 3059237130
939635303  | Marleny         | Álvarez Correa   | 3232410671
1089620113 | Myriam Yuliana  | Arango Narváez   | 3178267449
172555225  | Edilma Gabriela | Arias Álvarez    | 3213658326
550696749  | Viviana         | Arias Ramírez    | 3204850778
797017145  | Isabella        | Barrera Cabrera  | 3041744702
659942505  | Mariana         | Barrera Correa   | 3126485460
(10 rows)
```

Literal b)

b) Del mismo municipio, consulte los asociados (de cualquier género) que no tienen correo electrónico registrado.

Consulta — `ex_06_b.sql`

```
-- Exercise 06 - b
-- Farmers: farmers in Pamplona without registered email
SELECT
    T1.document_number AS document,
    T1.first_name AS first_name,
    T1.last_name AS last_name,
    T1.phone_number AS phone,
    T1.email AS email
FROM farm.farmers AS T1
WHERE T1.municipality_code = '54518'
  AND T1.email IS NULL;
```

Resultado obtenido en PostgreSQL

```
document   | first_name       | last_name        | phone      | email
-----------+------------------+------------------+------------+------
712758473  | Wilson           | López Villamizar | 3055094275 |
727945493  | Gloria           | Marín Rivera     | 3225571567 |
549054771  | María            | Burbano Duarte   | 3183590387 |
24999630   | Esperanza        | Romero Betancur  | 3029623613 |
914044314  | Ancízar Cristian | Polanía Delgado  | 3028825809 |
343256743  | Dora             | Hernández Correa | 3166719225 |
1148351420 | Inés             | Jiménez Jaimes   | 3104090231 |
1117597835 | Cecilia Lorena   | Díaz García      | 3207660684 |
749444168  | Emilio           | Delgado Molina   | 3118015821 |
940051048  | David Germán     | Romero Bonilla   | 3018117176 |
(10 rows)
```

Literal c)

c) Consulte los asociados del municipio 54518 cuya membresía esté inactiva, ordenados por fecha de afiliación del más antiguo al más reciente.

Consulta — `ex_06_c.sql`

```
-- Exercise 06 - c
-- Farmers: inactive membership in Pamplona ordered by affiliation date
SELECT
    T1.document_number AS document,
    T1.first_name AS first_name,
    T1.last_name AS last_name,
    T1.membership_date AS affiliation_date,
    T1.is_active AS is_active
FROM farm.farmers AS T1
WHERE T1.municipality_code = '54518'
  AND T1.is_active = FALSE
ORDER BY T1.membership_date ASC;
```

Resultado obtenido en PostgreSQL

```
document   | first_name     | last_name          | affiliation_date | is_active
-----------+----------------+--------------------+------------------+----------
230971064  | Manuel         | Agudelo Jiménez    | 1997-03-29       | false
534023693  | Myriam         | Valencia Moreno    | 1998-11-03       | false
33787285   | Rubén          | Ramos Chamorro     | 1999-03-02       | false
1135293825 | Abelardo       | Pérez Castro       | 2003-06-19       | false
478755763  | Nicolás        | Rivera Duque       | 2003-08-11       | false
160538872  | Liliana        | Mosquera Uribe     | 2007-09-27       | false
223974138  | Rodrigo Nelson | Cabrera Villamizar | 2009-05-02       | false
10424994   | Cristian       | Herrera Zuluaga    | 2010-04-28       | false
1015411133 | Santiago       | Bermúdez Ruiz      | 2010-11-30       | false
1090968710 | Ruth           | Rodríguez Pérez    | 2010-12-11       | false
(10 rows)
```

Ejercicio 7. BETWEEN y ORDER BY múltiple — Fincas de altura

Un comprador de cafés especiales busca fincas ubicadas a gran altitud en Pamplona, porque la altura se asocia con mejor calidad en taza.

Tabla: `farm.farms`

Literal a)

a) Consulte el nombre de la finca, la vereda, el área total y la altitud de las fincas del municipio 54518 ubicadas por encima de los 1800 m.s.n.m., con los alias `farm`, `rural_district`, `area_ha` y `altitude_masl`, ordenadas de mayor a menor altitud.

Consulta — `ex_07_a.sql`

```
-- Exercise 07 - a
-- Farms: altitude above 1800 masl in Pamplona
SELECT
    T1.farm_name AS farm,
    T1.rural_district AS rural_district,
    T1.total_area_ha AS area_ha,
    T1.altitude_masl AS altitude_masl
FROM farm.farms AS T1
WHERE T1.municipality_code = '54518'
  AND T1.altitude_masl > 1800
ORDER BY T1.altitude_masl DESC;
```

Resultado obtenido en PostgreSQL

```
farm          | rural_district | area_ha | altitude_masl
--------------+----------------+---------+--------------
Santa Clara   | La Legua       | 2.62    | 2300
El Guayabo    | El Rosal       | 1.21    | 2300
La Cascada    | Caima          | 2.49    | 2300
El Vergel     | Sabagua        | 1.11    | 2300
La Isabela    | García         | 1.36    | 2275
El Refugio    | Monteadentro   | 4.89    | 2242
Cafetal Viejo | Caima          | 5.29    | 2229
San José      | Alcaparral     | 3.12    | 2203
El Refugio    | Sabaneta Baja  | 1.93    | 2195
Villa Hermosa | Peñas          | 0.30    | 2181
(10 rows)
```

Literal b)

b) Del mismo municipio, consulte las fincas con altitud entre 1600 y 1800 m.s.n.m. (`BETWEEN`), ordenadas por vereda (ascendente) y luego por área (descendente).

Consulta — `ex_07_b.sql`

```
-- Exercise 07 - b
-- Farms: altitude BETWEEN 1600 and 1800 masl in Pamplona
SELECT
    T1.farm_name AS farm,
    T1.rural_district AS rural_district,
    T1.total_area_ha AS area_ha,
    T1.altitude_masl AS altitude_masl
FROM farm.farms AS T1
WHERE T1.municipality_code = '54518'
  AND T1.altitude_masl BETWEEN 1600 AND 1800
ORDER BY T1.rural_district ASC, T1.total_area_ha DESC;
```

Resultado obtenido en PostgreSQL

```
farm         | rural_district     | area_ha | altitude_masl
-------------+--------------------+---------+--------------
Los Pinos    | Alcaparral         | 1.67    | 1771
Miraflores   | Alcaparral         | 1.29    | 1636
Monteverde   | Alto Grande        | 2.59    | 1794
La Loma      | Alto Grande        | 2.09    | 1667
Bellavista   | Alto Grande        | 1.80    | 1704
El Refugio   | Alto Grande        | 1.23    | 1661
El Jardín    | Cabecera Municipal | 3.26    | 1620
El Edén      | Cabecera Municipal | 2.89    | 1603
La Milagrosa | Cabecera Municipal | 2.65    | 1643
La Divisa    | Cabecera Municipal | 1.32    | 1765
(10 rows)
```

Literal c)

c) Consulte las fincas del municipio 54518 que no tienen coordenadas geográficas registradas (latitud o longitud nulas).

Consulta — `ex_07_c.sql`

```
-- Exercise 07 - c
-- Farms: missing geographic coordinates in Pamplona
SELECT
    T1.farm_name AS farm,
    T1.rural_district AS rural_district,
    T1.latitude AS latitude,
    T1.longitude AS longitude
FROM farm.farms AS T1
WHERE T1.municipality_code = '54518'
  AND (T1.latitude IS NULL OR T1.longitude IS NULL);
```

Resultado obtenido en PostgreSQL

```
farm          | rural_district     | latitude | longitude
--------------+--------------------+----------+----------
El Recuerdo   | Alto Grande        |          |
La Esperanza  | El Palchal         |          |
Las Delicias  | Caima              |          |
Los Arrayanes | Cabecera Municipal |          |
El Brillante  | Peñas              |          |
El Porvenir   | Fontibón           |          |
La Florida    | Peñas              |          |
Los Arrayanes | Navarro            |          |
El Guayabo    | San Francisco      |          |
El Encanto    | Fontibón           |          |
(10 rows)
```

Ejercicio 8. IN y columnas calculadas — Entregas del día

El jefe de bodega revisa al cierre de mes las entregas recibidas el 30 de septiembre de 2026. El código de entrega incluye la fecha: `DLV-YYYYMMDD-NNNNNN`.

Tabla: `buy.deliveries`

Literal a)

a) Consulte el código, la finca, el punto de compra, la hora de recepción, el tipo de café y el peso bruto de las entregas de ese día (use `LIKE` sobre `delivery_id`), con los alias `delivery`, `farm_id`, `collection_point_id`, `received_at`, `coffee_type` y `gross_weight_kg`, ordenadas por hora de recepción.

Consulta — `ex_08_a.sql`

```
-- Exercise 08 - a
-- Deliveries: received on September 30, 2026 ordered by reception time
SELECT
    T1.delivery_id AS delivery,
    T1.farm_id AS farm_id,
    T1.collection_point_id AS collection_point_id,
    T1.delivered_at AS received_at,
    T1.coffee_type AS coffee_type,
    T1.gross_weight_kg AS gross_weight_kg
FROM buy.deliveries AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
ORDER BY T1.delivered_at ASC;
```

Resultado obtenido en PostgreSQL

```
delivery            | farm_id | collection_point_id | received_at         | coffee_type   |
gross_weight_kg
--------------------+---------+---------------------+---------------------+---------------+-------
---------
DLV-20260930-000001 | 121925  | 68                  | 2026-09-30 06:01:10 | dry_parchment | 51.53
DLV-20260930-000002 | 39639   | 48                  | 2026-09-30 06:03:32 | dry_parchment | 124.98
DLV-20260930-000003 | 228920  | 80                  | 2026-09-30 06:04:40 | wet_parchment | 112.33
DLV-20260930-000004 | 2649    | 90                  | 2026-09-30 06:07:26 | dry_parchment | 138.29
DLV-20260930-000005 | 163723  | 10                  | 2026-09-30 06:07:51 | dry_parchment | 256.42
DLV-20260930-000006 | 58408   | 170                 | 2026-09-30 06:11:03 | dry_parchment | 139.81
DLV-20260930-000007 | 201601  | 57                  | 2026-09-30 06:15:25 | dry_parchment | 404.97
DLV-20260930-000008 | 135490  | 53                  | 2026-09-30 06:16:53 | dry_parchment | 175.92
DLV-20260930-000009 | 108211  | 54                  | 2026-09-30 06:17:34 | dry_parchment | 56.80
DLV-20260930-000010 | 37288   | 67                  | 2026-09-30 06:20:30 | dry_parchment | 269.43
(10 rows)
```

Literal b)

b) Del mismo día, consulte las entregas rechazadas, agregando una columna calculada con el peso promedio por bulto (`gross_weight_kg / bag_count`) con el alias `avg_kg_per_bag`.

Consulta — `ex_08_b.sql`

```
-- Exercise 08 - b
-- Deliveries: rejected deliveries with calculated average kg per bag
SELECT
    T1.delivery_id AS delivery,
    T1.farm_id AS farm_id,
    T1.gross_weight_kg AS gross_weight_kg,
    T1.bag_count AS bag_count,
    (T1.gross_weight_kg / T1.bag_count) AS avg_kg_per_bag,
    T1.status AS status
FROM buy.deliveries AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.status = 'rejected';
```

Resultado obtenido en PostgreSQL

```
delivery            | farm_id | gross_weight_kg | bag_count | avg_kg_per_bag      | status
--------------------+---------+-----------------+-----------+---------------------+---------
DLV-20260930-000083 | 66435   | 89.95           | 2         | 44.9750000000000000 | rejected
DLV-20260930-000109 | 216626  | 311.88          | 5         | 62.3760000000000000 | rejected
DLV-20260930-000006 | 58408   | 139.81          | 3         | 46.6033333333333333 | rejected
DLV-20260930-000070 | 69157   | 147.13          | 3         | 49.0433333333333333 | rejected
DLV-20260930-000147 | 77310   | 66.42           | 1         | 66.4200000000000000 | rejected
DLV-20260930-000165 | 107038  | 348.59          | 6         | 58.0983333333333333 | rejected
DLV-20260930-000214 | 223116  | 177.63          | 3         | 59.2100000000000000 | rejected
DLV-20260930-000223 | 216415  | 338.32          | 7         | 48.3314285714285714 | rejected
DLV-20260930-000253 | 143807  | 253.15          | 5         | 50.6300000000000000 | rejected
(9 rows)
```

Literal c)

c) Del mismo día, consulte las entregas cuyo tipo de café sea `wet_parchment` o `cherry` (use `IN`), ordenadas por peso bruto de mayor a menor.

Consulta — `ex_08_c.sql`

```
-- Exercise 08 - c
-- Deliveries: coffee type wet_parchment or cherry using IN
SELECT
    T1.delivery_id AS delivery,
    T1.farm_id AS farm_id,
    T1.coffee_type AS coffee_type,
    T1.gross_weight_kg AS gross_weight_kg
FROM buy.deliveries AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.coffee_type IN ('wet_parchment', 'cherry')
ORDER BY T1.gross_weight_kg DESC;
```

Resultado obtenido en PostgreSQL

```
delivery            | farm_id | coffee_type   | gross_weight_kg
--------------------+---------+---------------+----------------
DLV-20260930-000081 | 19338   | cherry        | 475.13
DLV-20260930-000187 | 231839  | cherry        | 339.30
DLV-20260930-000105 | 13478   | wet_parchment | 277.63
DLV-20260930-000028 | 182397  | wet_parchment | 258.00
DLV-20260930-000133 | 44726   | cherry        | 254.83
DLV-20260930-000094 | 99627   | cherry        | 246.96
DLV-20260930-000129 | 89312   | wet_parchment | 206.71
DLV-20260930-000161 | 101318  | wet_parchment | 197.04
DLV-20260930-000046 | 85553   | wet_parchment | 186.73
DLV-20260930-000248 | 215660  | cherry        | 159.89
(10 rows)
```

Ejercicio 9. Rangos de calidad — Análisis de laboratorio

El laboratorio de calidad necesita identificar los mejores cafés analizados el 30 de septiembre de 2026 y las muestras que se salieron del rango de humedad aceptado (entre 10 % y 12 %).

Tabla: `buy.quality_analyses`

Literal a)

a) Consulte el código del análisis, la entrega, la humedad, el factor de rendimiento y el puntaje en taza de los análisis de ese día con `cup_score` mayor o igual a 85, con los alias `analysis`, `delivery`, `moisture_pct`, `yield_factor` y `cup_score`, ordenados por puntaje de mayor a menor.

Consulta — `ex_09_a.sql`

```
-- Exercise 09 - a
-- Quality analyses: cup score >= 85 on September 30, 2026
SELECT
    T1.analysis_id AS analysis,
    T1.delivery_id AS delivery,
    T1.moisture_pct AS moisture_pct,
    T1.yield_factor AS yield_factor,
    T1.cup_score AS cup_score
FROM buy.quality_analyses AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.cup_score >= 85
ORDER BY T1.cup_score DESC;
```

Resultado obtenido en PostgreSQL

```
analysis           | delivery            | moisture_pct | yield_factor | cup_score
-------------------+---------------------+--------------+--------------+----------
QA-20260930-000102 | DLV-20260930-000155 | 12.31        | 94.01        | 87.28
QA-20260930-000082 | DLV-20260930-000111 | 12.38        | 94.29        | 87.18
QA-20260930-000137 | DLV-20260930-000295 | 12.35        | 91.07        | 87.14
QA-20260930-000116 | DLV-20260930-000208 | 11.07        | 93.78        | 87.07
QA-20260930-000118 | DLV-20260930-000256 | 11.16        | 92.54        | 87.04
QA-20260930-000030 | DLV-20260930-000089 | 10.70        | 91.53        | 86.31
QA-20260930-000089 | DLV-20260930-000203 | 11.84        | 91.41        | 86.01
QA-20260930-000028 | DLV-20260930-000064 | 11.48        | 94.23        | 85.62
QA-20260930-000110 | DLV-20260930-000182 | 10.64        | 88.18        | 85.25
QA-20260930-000079 | DLV-20260930-000085 | 11.55        | 93.28        | 85.15
QA-20260930-000113 | DLV-20260930-000223 | 9.57         | 99.70        | 85.05
(11 rows)
```

Literal b)

b) Del mismo día, consulte los análisis cuya humedad esté fuera del rango 10 %–12 % (`NOT BETWEEN`), mostrando además el nombre del analista y las observaciones.

Consulta — `ex_09_b.sql`

```
-- Exercise 09 - b
-- Quality analyses: moisture percentage outside 10%-12% (NOT BETWEEN)
SELECT
    T1.analysis_id AS analysis,
    T1.delivery_id AS delivery,
    T1.moisture_pct AS moisture_pct,
    T1.analyst_name AS analyst_name,
    T1.observations AS observations
FROM buy.quality_analyses AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.moisture_pct NOT BETWEEN 10 AND 12;
```

Resultado obtenido en PostgreSQL

```
analysis           | delivery            | moisture_pct | analyst_name     | observations
-------------------+---------------------+--------------+------------------+----------------------
------------------------
QA-20260930-000026 | DLV-20260930-000005 | 12.19        | Wilson Galvis    |
QA-20260930-000018 | DLV-20260930-000006 | 17.24        | Nicolás Serrano  | Café con humedad
alta, requiere más secado.
QA-20260930-000017 | DLV-20260930-000012 | 12.15        | Natalia Trujillo |
QA-20260930-000033 | DLV-20260930-000027 | 12.41        | Sara Sánchez     |
QA-20260930-000013 | DLV-20260930-000038 | 12.28        | Adriana Losada   |
QA-20260930-000005 | DLV-20260930-000040 | 12.22        | Iván Castro      |
QA-20260930-000027 | DLV-20260930-000043 | 12.07        | Germán González  | Caficultor solicita
contramuestra.
QA-20260930-000144 | DLV-20260930-000289 | 12.28        | Ancízar Delgado  |
QA-20260930-000137 | DLV-20260930-000295 | 12.35        | Sara Sánchez     |
QA-20260930-000023 | DLV-20260930-000046 | 12.23        | Alejandra Torres | Café de buena
presentación, sin olores extraños.
(10 rows)
```

Literal c)

c) Del mismo día, consulte los análisis no aceptados que sí tengan observaciones registradas, ordenados por porcentaje de defectos de mayor a menor.

Consulta — `ex_09_c.sql`

```
-- Exercise 09 - c
-- Quality analyses: unaccepted samples with registered notes
SELECT
    T1.analysis_id AS analysis,
    T1.delivery_id AS delivery,
    T1.is_accepted AS is_accepted,
    T1.defects_pct AS defects_pct,
    T1.observations AS observations
FROM buy.quality_analyses AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.is_accepted = FALSE
  AND T1.observations IS NOT NULL
ORDER BY T1.defects_pct DESC;
```

Resultado obtenido en PostgreSQL

```
analysis           | delivery            | is_accepted | defects_pct | observations
-------------------+---------------------+-------------+-------------+---------------------------
---------------
QA-20260930-000104 | DLV-20260930-000214 | false       | 19.54       | Olor a fermento en la
muestra.
QA-20260930-000068 | DLV-20260930-000165 | false       | 17.29       | Muestra tomada por
duplicado.
QA-20260930-000119 | DLV-20260930-000253 | false       | 15.59       | Exceso de impurezas y
pasilla.
QA-20260930-000086 | DLV-20260930-000147 | false       | 13.21       | Olor a fermento en la
muestra.
QA-20260930-000113 | DLV-20260930-000223 | false       | 12.94       | Café sobresecado, grano
quebradizo.
QA-20260930-000022 | DLV-20260930-000070 | false       | 9.75        | Café con humedad alta,
requiere más secado.
QA-20260930-000018 | DLV-20260930-000006 | false       | 7.88        | Café con humedad alta,
requiere más secado.
(7 rows)
```

Ejercicio 10. Columnas generadas — Liquidaciones pendientes

Tesorería debe programar los pagos de las liquidaciones generadas el 30 de septiembre de 2026. Recuerde que `gross_amount` y `net_amount` son columnas `GENERATED` que la base calcula sola.

Tabla: `pay.settlements`

Literal a)

a) Consulte el código de la liquidación, la entrega, el peso neto, el precio aplicado, el valor bruto, las deducciones y el valor neto de las liquidaciones de ese día, con los alias `settlement`, `delivery`, `net_weight_kg`, `price_per_kg`, `gross_cop`, `deductions_cop` y `net_cop`.

Consulta — `ex_10_a.sql`

```sql
-- Exercise 10 - a
-- Settlements: all settlements from September 30, 2026
SELECT
    T1.settlement_id AS settlement,
    T1.delivery_id AS delivery,
    T1.net_weight_kg AS net_weight_kg,
    T1.price_per_kg_applied AS price_per_kg,
    T1.gross_amount AS gross_cop,
    T1.deductions_amount AS deductions_cop,
    T1.net_amount AS net_cop
FROM pay.settlements AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%';
```

Resultado obtenido en PostgreSQL

```
settlement          | delivery            | net_weight_kg | price_per_kg | gross_cop   |
deductions_cop | net_cop
--------------------+---------------------+---------------+--------------+-------------+----------
------+------------
STL-20261001-000006 | DLV-20260930-000003 | 109.68        | 22083.00     | 2422063.44  | 0.00
| 2422063.44
STL-20261002-000030 | DLV-20260930-000004 | 137.35        | 23052.00     | 3166192.20  | 52622.11
| 3113570.09
STL-20261001-000060 | DLV-20260930-000005 | 251.37        | 21052.00     | 5291841.24  | 0.00
| 5291841.24
STL-20261002-000005 | DLV-20260930-000008 | 174.75        | 23230.00     | 4059442.50  | 224252.16
| 3835190.34
STL-20261002-000004 | DLV-20260930-000010 | 265.10        | 21358.00     | 5662005.80  | 314758.77
| 5347247.03
STL-20261003-000006 | DLV-20260930-000012 | 47.08         | 21627.00     | 1018199.16  | 0.00
| 1018199.16
STL-20261002-000043 | DLV-20260930-000017 | 535.29        | 21294.00     | 11398465.26 | 634980.18
| 10763485.08
STL-20261002-000018 | DLV-20260930-000020 | 113.45        | 21439.00     | 2432254.55  | 0.00
| 2432254.55
STL-20261003-000001 | DLV-20260930-000023 | 219.26        | 22645.00     | 4965142.70  | 0.00
| 4965142.70
STL-20261002-000006 | DLV-20260930-000025 | 173.32        | 21155.00     | 3666584.60  | 0.00
| 3666584.60
(10 rows)
```

Literal b)

b) Del mismo día, consulte solo las liquidaciones en estado `pending`, ordenadas por valor neto de mayor a menor.

Consulta — `ex_10_b.sql`

```
-- Exercise 10 - b
-- Settlements: pending settlements ordered by net amount descending
SELECT
    T1.settlement_id AS settlement,
    T1.delivery_id AS delivery,
    T1.net_amount AS net_cop,
    T1.status AS status
FROM pay.settlements AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.status = 'pending'
ORDER BY T1.net_amount DESC;
```

Resultado obtenido en PostgreSQL

```
settlement          | delivery            | net_cop    | status
--------------------+---------------------+------------+--------
STL-20261001-000040 | DLV-20260930-000256 | 6094824.45 | pending
STL-20261001-000057 | DLV-20260930-000105 | 6051013.94 | pending
STL-20261003-000007 | DLV-20260930-000116 | 5349871.04 | pending
STL-20261002-000004 | DLV-20260930-000010 | 5347247.03 | pending
STL-20260930-000069 | DLV-20260930-000204 | 5013706.66 | pending
STL-20261003-000001 | DLV-20260930-000023 | 4965142.70 | pending
STL-20261002-000042 | DLV-20260930-000152 | 4693178.85 | pending
STL-20261003-000012 | DLV-20260930-000286 | 4135855.75 | pending
STL-20261002-000033 | DLV-20260930-000189 | 4023811.68 | pending
STL-20261001-000033 | DLV-20260930-000179 | 3978665.28 | pending
(10 rows)
```

Literal c)

c) Del mismo día, consulte las liquidaciones sin deducciones, agregando una columna calculada que compruebe el valor bruto (`net_weight_kg * price_per_kg_applied`) con el alias `check_gross_cop`. Compare esa columna con `gross_amount` y explique con sus propias palabras por qué coinciden.

Consulta — `ex_10_c.sql`

```
-- Exercise 10 - c
-- Settlements: settlements without deductions and manual gross validation
SELECT
    T1.settlement_id AS settlement,
    T1.delivery_id AS delivery,
    T1.net_weight_kg AS net_weight_kg,
    T1.price_per_kg_applied AS price_per_kg,
    T1.gross_amount AS gross_cop,
    (T1.net_weight_kg * T1.price_per_kg_applied) AS check_gross_cop
FROM pay.settlements AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
  AND T1.deductions_amount = 0;
```

Resultado obtenido en PostgreSQL

```
settlement          | delivery            | net_weight_kg | price_per_kg | gross_cop  |
check_gross_cop
--------------------+---------------------+---------------+--------------+------------+-----------
-----
STL-20261001-000006 | DLV-20260930-000003 | 109.68        | 22083.00     | 2422063.44 |
2422063.4400
STL-20261001-000060 | DLV-20260930-000005 | 251.37        | 21052.00     | 5291841.24 |
5291841.2400
STL-20261003-000006 | DLV-20260930-000012 | 47.08         | 21627.00     | 1018199.16 |
1018199.1600
STL-20261002-000018 | DLV-20260930-000020 | 113.45        | 21439.00     | 2432254.55 |
2432254.5500
STL-20261003-000001 | DLV-20260930-000023 | 219.26        | 22645.00     | 4965142.70 |
4965142.7000
STL-20261002-000006 | DLV-20260930-000025 | 173.32        | 21155.00     | 3666584.60 |
3666584.6000
STL-20261001-000058 | DLV-20260930-000027 | 180.33        | 22112.00     | 3987456.96 |
3987456.9600
STL-20261002-000024 | DLV-20260930-000029 | 228.30        | 21942.00     | 5009358.60 |
5009358.6000
STL-20261002-000019 | DLV-20260930-000111 | 287.72        | 21454.00     | 6172744.88 |
6172744.8800
STL-20261001-000023 | DLV-20260930-000112 | 46.76         | 22110.00     | 1033863.60 |
1033863.6000
(10 rows)
```

Conclusión (explicación del literal c)

> Coinciden porque la columna gross_amount es una columna calculada automáticamente por PostgreSQL: la base de datos realiza por debajo exactamente la misma multiplicación entre el peso de la entrega (net_weight_kg) y el valor por kilo (price_per_kg_applied). Como en este ejercicio filtramos únicamente las liquidaciones que no tienen descuentos o deducciones, la operación manual que hacemos en el SELECT da exactamente el mismo resultado que la columna almacenada.